

Learning Journey Roadmap



START

└─ Topic 1: Introduction to Echo and Comments



└─ Topic 2: Variables in Bash



└─ Topic 3: Data Types in Bash



└─ Topic 4: User Input



└─ Topic 5: Arithmetic Operations



└─ Topic 6: String Operations



└─ Topic 7: Variable Updates and Shortcuts



DECISION MAKING



└─ Topic 8: Relational Operations



└─ Topic 9: Conditional Statements



└─ Topic 10: Logical Operations



AUTOMATION



└─ Topic 11: Arrays and For Loops



└─ Topic 12: While Loops



└─ Topic 13: Loop Control



└─ Topic 14: Case Statement



FINAL PROJECTS

└─ Expense Tracker

└─ Login System

└─ Traffic Light System

└─ Running Balance Calculator

└─ Sales Tracker

└─ Sales Tracker v2



BASH SCRIPTING FUNDAMENTALS COMPLETED

Bash Scripting Module:

Module Overview

This comprehensive module covers the fundamentals of Bash scripting, designed for beginners with no prior experience. Each topic includes theoretical notes, practical examples with filenames, and hands-on lab exercises.

Topic 1: Introduction to Echo and Comments

Learning Objectives

- Understand what Bash scripting is and its uses
- Create and execute your first Bash script
- Use `echo` command for output
- Write single-line and multi-line comments

Sample Scripts

01_echo.sh - Your First Script

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

This script demonstrates a basic Bash script with a shebang line, a comment, and an echo statement.

02_echo_multiple_line.sh - Multiple Echo Statements

```
#!/bin/bash
# Printing Multiple
echo "Hello, World!"
echo "Welcome to Bash Scripting!"
```

This shows how to print on multiple lines using separate echo commands.

03_echo_multiple_line.sh - Multiple Arguments in One Echo

```
#!/bin/bash
# Print multiple strings in one line
echo "I" "am" "happy"
```

This demonstrates passing multiple arguments to echo, which prints them on the same line with spaces.

01_comment.sh - Single-Line Comments

```
#!/bin/bash
# Practice for printing in multiple lines
echo "Hello, I am new to Bash scripting."
echo "Lets learn together!"
```

Shows the use of single-line comments to document code.

02_multiline_comment.sh - Multi-Line Comments

```
#!/bin/bash

:'
This is a very simple application to test the print function.
Remember that strings need double or single quotes!
'

echo "Hello, I am new to Bash Scripting"
echo "Lets learn together!"
```

Demonstrates multi-line comments using the `:'` syntax.

Lab Exercises

Lab 1.1: Hello World Script

File: 01_hello_v1.sh

```
#!/bin/bash
# TODO: Print the following in the console:
# Hello, My name is Yourname
# Welcome to Bash Scripting!

# TO DO: Print the following in the console using echo command
your code here...
```

Topic 2: Variables in Bash

Learning Objectives

- Declare and use variables
- Understand variable naming conventions
- Perform variable reassignment
- Use variable expansion

Sample Scripts

01_variable.sh - Basic Variable Declaration

```
#!/bin/bash
message="Test Message"
```

Shows the simplest form of variable declaration.

02_variable_echo.sh - Echoing Variables

```
#!/bin/bash
message="Test Message"
echo "$message"
```

Demonstrates how to print the value of a variable using `$` prefix.

03_variable_text.sh - Variable vs Literal

```
#!/bin/bash
# echo the value "Test Message"
message="Test Message"
echo "$message"

# echo the literal "message"
echo "message"
```

Shows the difference between echoing a variable value and a literal string.

04_variable_nickname.sh - Variables for Long Values

```
#!/bin/bash
# Variables are great for using long values
name="José Protacio Rizal Mercado y Alonso Realonda"

echo "$name"
```

Demonstrates using variables to store and reuse long/complex values.

05_variable_reassignment.sh - Reassigning Variables

```
#!/bin/bash
result=9999
echo "$result"

result=1
echo "$result"
```

Shows how variables can be reassigned with new values.

06_variable_evaluation.sh - Variable Evaluation

```
#!/bin/bash
# age -> 30
# age = 30 + 1

declare -i age=30
age=age+1
echo "Age is: $age"
```

Demonstrates evaluation of variables in arithmetic operations.

07_variable_reassignment.sh - Game Level Example

```
#!/bin/bash
declare -i level=1
echo "Level: $level"

# Player gains XP
level=level+1
echo "Leveled up! Level: $level"
```

Practical example showing variable updates in a gaming context.

08_variable_reassignment.sh - Battery Status Example

```
#!/bin/bash
battery=100
echo "Battery: $battery"

# Opened Chrome with 10 tabs
battery=$((battery - 40))
echo "After Chrome: $battery"

# Plugged in charger
battery=$((battery + 20))
echo "Charger inserted: $battery"
```

Practical example showing variable updates with arithmetic.

Lab Exercises

Lab 2.1: Counter Operations

File: 02_counter.sh

```
#!/bin/bash
# TO DO: Create a counter variable and perform the following operations:

# variable to hold the counter value
counter=0
echo "Counter: $counter"

# Add one to the counter
your code here
echo "Counter: $counter"

# Multiply the counter by 10
your code here
echo "Counter: $counter"

# Decrease the counter by 4
your code here
echo "Counter: $counter"
```

Topic 3: Data Types in Bash

Learning Objectives

- Work with strings
- Handle integer variables
- Understand floating-point numbers
- Use boolean values

Sample Scripts

01_strings.sh - String Data Type

```
#!/bin/bash
empty_string_a=""
echo "Empty string: $empty_string_a"

empty_string_b=""
echo "Empty string: $empty_string_b"

quote="I am a little teapot, short and spout."
echo "Quote: $quote"
```

Demonstrates different ways to create and display strings.

02_integer.sh - Integer Data Type

```
#!/bin/bash
declare -i savings=0
echo "Savings: $savings"

declare -i balance=-100
echo "Balance: $balance"

declare -i high_score=111222333
echo "High Score: $high_score"
```

Shows how to declare and use integer variables.

03_float.sh - Float Values (as Strings)

```
#!/bin/bash
temperature_celsius=0.0
echo "Temperature celsius: $temperature_celsius"

growth_rate=-0.56
echo "Growth rate: $growth_rate"

avogadros_number=6.022e+23
echo "Avogadros number: $avogadros_number"
```

Bash doesn't natively support floats; they're stored as strings.

04_bool.sh - Boolean Values

```
#!/bin/bash
is_raining=true
echo "Is raining: $is_raining"

exit_program=false
echo "Exit program: $exit_program"
```

Demonstrates using strings to represent boolean values.

Lab Exercises

Lab 3.1: Wishlist Application

File: 03_wishlist.sh

```
#!/bin/bash

# Fill in the variables based on the item you want to buy
name="" # What is the name of the item?
price="" # How much is the item?
organic="" # Is it organic?

# Print each information one line at a time
your code here
your code here
your code here
```

Topic 4: User Input

Learning Objectives

- Read user input using `read`
- Convert input to integers
- Handle different data types

Sample Scripts

01_input.sh - Basic User Input

```
#!/bin/bash
read -p "Enter your input: " user_input
echo "You entered: $user_input"
```

Demonstrates using `read -p` to get user input with a prompt.

02_input_conversion.sh - Input Conversion

```
#!/bin/bash
# declare as integer
declare -i number1
declare -i number2
declare -i total

# Doing actual addition
read -p "Enter first number: " number1
read -p "Enter second number: " number2
total=number1+number2
echo "Total: $total"
```

Shows how to convert input to integers using `declare -i`.

Lab Exercises

Lab 4.1: Expense Tracker

File: 04_expense_tracker.sh

```
#!/bin/bash

# Ask the user for three values
your code here
your code here
your code here

# Print each information one line at a time
echo
echo "Your expense list are:"
your code here
your code here
your code here
```

Lab 4.2: Expense Tracker with Calculations

File: 05_expense_tracker_v2.sh

```
#!/bin/bash

# Ask the user for three values
read -p "Enter expense_1: " expense_1
read -p "Enter expense_2: " expense_2
read -p "Enter expense_3: " expense_3

# Print each information one line at a time
your code here
your code here
your code here

# Calculate the sum of the numbers
your code here

# Print the total expense
your code here
```

Topic 5: Arithmetic Operations

Learning Objectives

- Perform addition, subtraction, multiplication, and division
- Use arithmetic expansion `$(...)`
- Use `bc` for floating-point calculations
- Understand operator precedence

Sample Scripts

01_addition.sh - Addition

```
#!/bin/bash
read -p "Enter first number: " number1
read -p "Enter second number: " number2

# $((...)) # arithmetic expansion
echo "Sum: $((number1 + number2))"
```

Demonstrates addition using arithmetic expansion.

02_subtraction.sh - Subtraction

```
#!/bin/bash
read -p "Enter first number: " number1
read -p "Enter second number: " number2

# $((...)) # arithmetic expansion
echo "Difference: $((number1 - number2))"
```

Shows subtraction with arithmetic expansion.

03_multiplication.sh - Multiplication

```
#!/bin/bash
read -p "Enter first number: " number1
read -p "Enter second number: " number2

# $((...)) # arithmetic expansion
echo "Product: $((number1 * number2))"
```

Demonstrates multiplication.

04_division.sh - Division with bc

```
#!/bin/bash
read -p "Enter first number: " number1
read -p "Enter second number: " number2

# $(...) # command substitution
echo "Division: $(echo "$number1 / $number2" | bc)"

# scale=2 means show 2 digits after the decimal point.
echo "Division: $(echo "scale=2; $number1 / $number2" | bc)"
```

Shows division using `bc` for floating-point results.

05_order_of_operation.sh - Operator Precedence

```
#!/bin/bash
result=$(echo "3 + 5 * 2 - 8 / 4" | bc)
echo "$result"

# Enforce using parentheses PEMDAS
result=$(echo "3 + (5 * 2) - (8 / 4)" | bc)
echo "$result"
```

Demonstrates operator precedence and use of parentheses.

Lab Exercises

Lab 5.1: Price Post

File: 06_price_post.sh

```
#!/bin/bash
```

```
# Post: Latte (3.5)
```

```
your code here
```

```
# Post: Espresso (2.75)"
```

```
your code here
```

```
# Post: Cappuccino (4.0)
```

```
your code here
```

Lab 5.2: Sales Tracker

File: 07_sales_tracker.sh

```
#!/bin/bash
```

```
# Ask the cost and quantity for three separate items
```

```
your code here
```

```
your code here
```

```
your code here
```

```
# Calculate the total
```

```
your code here
```

```
# print the total
```

```
your code here
```

Topic 6: String Operations

Learning Objectives

- Concatenate strings
- Work with string manipulation

Sample Scripts

01_string_concatenation.sh - String Concatenation

```
#!/bin/bash
```

```
echo "Hello World"
```

```
# string concatenation
```

```
echo "Hello" "World"
```

Topic 7: Variable Updates and Shortcuts

Learning Objectives

- Update variables using arithmetic
- Use shortcut operators
- Apply updates in practical scenarios

Sample Scripts

01_update_shortcut.sh - Shortcut Operators

```
#!/bin/bash
result=10

# Original
# result=$((result + 5))

# Shortcut
((result += 5))

# Original
# result=$((result * 10))

# Shortcut
((result *= 10))

echo "$result"
```

Demonstrates shortcut operators for arithmetic operations.

02_update_example.sh - Level Up Example

```
#!/bin/bash
level=1
echo "Level: $level"

# Player gains XP
# Previously: level=$((level + 1))
((level += 1))

echo "Leveled up! Level: $level"
```

Practical example using shortcut operators.

03_update_example.sh - Battery Example

```
#!/bin/bash
battery=100
echo "Battery: $battery"

# Opened Chrome with 10 tabs
# Previously: battery=$((battery - 40))
((battery -= 40))

echo "After Chrome: $battery"

# Plugged in charger
# Previously: battery=$((battery + 20))
((battery += 20))

echo "Charger inserted: $battery"
```

Another practical example with shortcut operators.

Topic 8: Relational Operations

Learning Objectives

- Compare numbers using relational operators
- Use arithmetic evaluation with relational operators
- Chain multiple relational conditions
- Understand equality and inequality comparisons

Sample Scripts

01_less_than.sh - Less Than Comparison

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
less_than=$(( (first_number < second_number) )) && echo True || echo False

echo "Less than: $less_than"
```

Demonstrates the less than (<) operator in arithmetic evaluation.

02_less_than_equal.sh - Less Than or Equal

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
less_than_equal=$(( (first_number <= second_number) )) && echo True || echo False

echo "Less than equal: $less_than_equal"
```

Shows the less than or equal (<=) operator.

03_greater_than.sh - Greater Than Comparison

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
greater_than=$(( first_number > second_number )) && echo True || echo False

echo "Greater than: $greater_than"
```

Demonstrates the greater than (>) operator.

04_greater_than_equal.sh - Greater Than or Equal

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
greater_than_equal=$(( first_number >= second_number )) && echo True || echo False

echo "Greater than equal: $greater_than_equal"
```

Shows the greater than or equal (>=) operator.

05_chained_relational.sh - Chained Relational Operations

```
#!/bin/bash
read -p "Enter number: " x
echo

echo "Exclusive Range"
(( x > 3 && x < 20 )) && echo True || echo False
echo

echo "Equal or Greater than 3 and Less than 20"
(( x >= 3 && x < 20 )) && echo True || echo False
echo

echo "Greater than 3 and Less than or Equal to 20"
(( x > 3 && x <= 20 )) && echo True || echo False
echo

echo "Inclusive Range"
(( x >= 3 && x <= 20 )) && echo True || echo False
```

Demonstrates chaining multiple relational operations with logical AND.

06_equality.sh - Equality Comparison

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
equal=$(( first_number == second_number )) && echo True || echo False

echo "Equal: $equal"
```

Shows the equality (==) operator.

07_unequality.sh - Inequality Comparison

```
#!/bin/bash
read -p "Enter the first number: " first_number
read -p "Enter the second number: " second_number

# arithmetic evaluation
not_equal=$(( (first_number != second_number) )) && echo True || echo False

echo "Not equal: $not_equal"
```

Demonstrates the inequality (!=) operator.

08_equality_sample.sh - Perfect Score Check

```
#!/bin/bash
read -p "Enter score: " score
perfect_score=100

# arithmetic evaluation
has_perfect_score=$(( (score == perfect_score) )) && echo True || echo False

echo "You got a perfect score: $has_perfect_score"
```

Practical example using equality comparison.

Lab Exercises

Lab 8.1: Height Requirement Check

File: 09_height_requirement.sh

```
#!/bin/bash
# TO DO: Check if the user is tall enough to enter the ride

# Height requirement for the ride
minimum_height=138

# TO DO: Ask the user for their height
your code here

# TO DO: Determine if the user can enter the ride
your code here
your code here
your code here
your code here

# TO DO: Print the result
your code here
```

Lab 8.2: Valid Score Check

File: 10_valid_score.sh

```
#!/bin/bash
# TO DO: Check if the user has entered a valid score

# Range minimum and maximum bounds
min_number=0
max_number=100

# TO DO: Ask the user to enter a score
your code here

# TO DO: Determine if the score is valid
your code here
your code here
your code here
your code here

# TO DO: Print the result
your code here
```

Topic 9: Conditional Statements

Learning Objectives

- Use if statements for conditional execution
- Implement if-else for binary decisions
- Use if-elif-else for multiple conditions
- Understand the difference between multiple ifs and if-elif

Sample Scripts

01_if_statement.sh - Basic If Statement

```
#!/bin/bash
read -p "Login: " login_input

# string comparisons
if [[ "$login_input" == "Yes" ]]; then
    echo "Welcome"
    echo "Back"
fi

echo "End"
```

Demonstrates a basic if statement with string comparison.

02_if_elif_statement_example.sh - If-Elif Chain

```
#!/bin/bash
read -p "You said: " you_said

if [[ "$you_said" == "Wish" ]]; then
    echo "107.5"
elif [[ "$you_said" == "Hello" ]]; then
    echo "...it's me"
elif [[ "$you_said" == "Jopay" ]]; then
    echo "...kamusta ka na"
elif [[ "$you_said" == "Black Pink" ]]; then
    echo "...in your area"
fi
```

Shows an if-elif chain for multiple conditions.

03_if_elif_statement_example.sh - Battery Status

```
#!/bin/bash
read -p "Battery percentage: " battery

if (( battery >= 80 )); then
    echo "Full Battery"
elif (( battery >= 40 )); then
    echo "Good Battery"
elif (( battery >= 15 )); then
    echo "Low Battery"
elif (( battery > 0 )); then
    echo "Critically Low Battery"
fi
```

Practical example with numeric comparisons.

04_if_elif_else_statement.sh - If-Elif-Else

```
#!/bin/bash
read -p "You said: " you_said

if [[ "$you_said" == "Wish" ]]; then
    echo "107.5"
elif [[ "$you_said" == "Hello" ]]; then
    echo "...it's me"
elif [[ "$you_said" == "Jopay" ]]; then
    echo "...kamusta ka na"
elif [[ "$you_said" == "Black Pink" ]]; then
    echo "...in your area"
else
    echo "I don't know that song!"
fi
```

Adds an else clause for unmatched conditions.

05_multiple_if.sh - Multiple If Statements

```
#!/bin/bash
grade=85

if (( grade >= 90 )); then
    echo "A"
```



```

fi

if (( grade >= 80 )); then
    echo "B"
fi

if (( grade >= 70 )); then
    echo "C"
fi

```

Shows multiple independent if statements vs if-elif.

06_if_elif_example.sh - If-Elif for Grades

```

#!/bin/bash
grade=85

if (( grade >= 90 )); then
    echo "A"
elif (( grade >= 80 )); then
    echo "B"
elif (( grade >= 70 )); then
    echo "C"
fi

```

Proper grade checking using if-elif.

07_if_else_example.sh - Age Verification

```

#!/bin/bash
read -p "Enter age: " age

if (( age >= 18 )); then
    echo "Old enough to watch movie"
else
    echo "Too young to watch movie"
fi

```

Simple if-else for age verification.

08_if_else_example.sh - Balance Check

```

#!/bin/bash
balance=150
price=200

if (( balance >= price )); then
    echo "Payment successful"
else
    echo "Insufficient funds"
fi

```

Practical if-else for financial validation.

Lab Exercises

Lab 9.1: Login System

File: 11_login_system.sh

```
#!/bin/bash
# TO DO: Create a simple login system

# Expected username and password (you can change the value)
correct_username="user"
correct_password="pass"

# TO DO: Ask the user to enter their username and password
your code here
your code here

# TO DO: Determine if the credentials are valid
your code here
your code here
your code here
your code here
your code here
```

Lab 9.2: Traffic Light System

File: 12_traffic_lights.sh

```
#!/bin/bash
# TO DO: Ask the user to enter a color

# TO DO: Read the color input from the user
you

# Print the following depending on the color input
# "green" -> print "Go"
# "yellow" -> print "Wait..."
# "red" -> print "Stop"
# Everything else -> print "Malfunction"

# TO DO: Determine the appropriate action based on the color input
your code here
your code here
your code here
your code here
your code here
```

Topic 10: Logical Operations

Learning Objectives

- Use AND (&&) for multiple conditions
- Use OR (||) for alternative conditions
- Use NOT (!) for negation
- Combine logical operators for complex conditions

Sample Scripts

01_and_operation_example.sh - AND with Strings

```
#!/bin/bash
applied=true
has_skill=true
has_experience=true

if [[ "$applied" == true && "$has_skill" == true && "$has_experience" == true ]]; then
    echo "You're hired"
fi
```

Demonstrates AND operation with string comparisons.

02_and_operation_example.sh - AND with Numbers

```
#!/bin/bash
read -p "Enter money: " money
read -p "Enter stock: " stock

if (( money >= 100 && stock > 0 )); then
    echo "You can buy the item!"
else
    echo "You can't buy the item"
fi
```

Shows AND operation with numeric comparisons.

03_or_operation_example.sh - OR with Multiple Conditions

```
#!/bin/bash
raining=false
cold=false
trendy=true

if [[ "$raining" == true || "$cold" == true || "$trendy" == true ]]; then
    echo "Wear a jacket"
fi
```

Demonstrates OR operation with multiple conditions.

04_or_operation_example.sh - OR with Case Insensitivity

```
#!/bin/bash
read -p "Continue? " response

if [[ "$response" == "yes" || "$response" == "YES" ]]; then
    echo "We will continue!"
fi
```

Shows OR for case-insensitive input handling.

05_not_operation_example.sh - NOT Operation

```
#!/bin/bash
logged_in=false

if ! $logged_in; then
    echo "Access denied"
fi
```

Demonstrates the NOT (!) operator for negation.

Topic 11: Arrays and For Loops

Learning Objectives

- Create and access arrays
- Iterate over arrays using for loops
- Use brace expansion for ranges
- Use C-style for loops
- Aggregate and count data in loops

Sample Scripts

01_array.sh - Array Basics

```
#!/bin/bash
items=("milk" "egg" "ice")

echo "${items[0]} ${items[1]} ${items[2]}"
echo "${items[@]}"
```

Demonstrates array creation and accessing elements.

02_for_in_loop_array.sh - For Loop with Arrays

```
#!/bin/bash
items=("milk" "egg" "ice")

for item in "${items[@]}"
do
    echo "$item"
done
```

Shows iterating over array elements.

03_for_in_loop_list.sh - For Loop with List

```
#!/bin/bash
for notification in "Battery low" "New message" "New Update"
do
    echo "Alert: $notification"
done
```

Demonstrates for loop with a list of strings.

04_forr_in_loop_brace_expansion.sh - Brace Expansion

```
#!/bin/bash
for item in {0..2}
do
    echo "$item: This will be repeated"
done
```

Shows for loop with brace expansion.

05_forr_in_loop_C-style.sh - C-Style For Loop

```
#!/bin/bash
for ((item=0; item<=3; item++))
do
    echo "$item: This will be repeated"
done
```

Demonstrates C-style for loop syntax.

08_aggregation.sh - Aggregating with For Loop

```
#!/bin/bash
expenses=(1500 2300 1200)
total=0

for amount in "${expenses[@]}"
do
    total=$((total + amount))
done
```

```
echo "Total expenses: $total"
```

Shows how to aggregate values using a for loop.

09_counting.sh - Counting with For Loop

```
#!/bin/bash
grades=(85 72 49 90 66 59)
passing=0

for grade in "${grades[@]}"
do
    if (( grade >= 60 )); then
        passing=$((passing + 1))
    fi
done
```

```
echo "Passing: $passing"
```

Demonstrates counting with conditional logic in a loop.

Lab Exercises

Lab 11.1: Bookmarks List

File: 13_bookmarks.sh

```
#!/bin/bash
#TO DO: Print a list of your favorite websites

# Define a list of your favorite websites
websites=("google.com" "facebook.com" "yahoo.com" "youtube.com" "eugene.com")

# TO DO: Print each website in the list using for loop array
your code here
your code here
your code here
your code here
```

Lab 11.2: Search in Array

File: 15_search.sh

```
#!/bin/bash
# TO DO: Search for an item in a list

# Define a list of items
items=("rice" "noodles" "toyo" "spam" "coffee")
item_to_find="spam"

# TO DO: Search for the item in the list
your code here
your code here
your code here
```

Topic 12: While Loops

Learning Objectives

- Use while loops for condition-based iteration
- Implement infinite loops with break conditions
- Create interactive command loops

Sample Scripts

01_while_loop_example.sh - Password Validation

```
#!/bin/bash
read -p "Password: " password

while [[ "$password" != "pass" ]]
do
    read -p "Password: " password
done
```

Demonstrates a while loop for password validation.

02_while_loop_example.sh - Interactive Command Loop

```
#!/bin/bash
while true
do
    read -p "Provide command: " command

    if [[ "$command" == "command 1" ]]; then
        echo "command 1 done"
    elif [[ "$command" == "command 2" ]]; then
        echo "command 2 done"
    elif [[ "$command" == "command 3" ]]; then
        echo "command 3 done"
    elif [[ "$command" == "exit" ]]; then
        break
    fi
done
```

Shows an infinite while loop with break condition.

Lab Exercises

Lab 12.1: Running Balance Calculator

File: 14_running_balance.sh

```
#!/bin/bash
# TO DO: Create a running balance calculator

# Initialize total and running variables
total=0
running=true

# TO DO: Create a loop that will continue until the user exits
while $running
do
    read -p "Provide command: " command

    if [ "$command" = "add" ]; then

        # TO DO: Ask for number
        your code here

        # TO DO: Add that number to the total
        your code here

        # TO DO: Print the current total
        your code here

    elif [ "$command" = "sub" ]; then

        # TO DO: Ask for number
        your code here

        # TO DO: Subtract that number from the total
        your code here

        # TO DO: Print the current total
        your code here

    elif [ "$command" = "exit" ]; then
        echo
        echo "bye!"
        running=false
    fi
done
```


Topic 13: Loop Control

Learning Objectives

- Use break to exit loops
- Use continue to skip iterations
- Control loop flow effectively

Sample Scripts

01_break_example.sh - Break in Loop

```
#!/bin/bash
for item in {0..99}
do
    echo "$item"

    if (( item == 3 )); then
        break
    fi
done
```

Demonstrates using break to exit a loop.

02_break_example.sh - Break in Password Attempts

```
#!/bin/bash
max_attempt=3
correct_password="pass"

for ((attempt=0; attempt<max_attempt; attempt++))
do
    read -p "Enter password: " password

    if [[ "$password" == "$correct_password" ]]; then
        echo "Access granted"
        break
    fi
done
```

Shows break used in a password attempt loop.

03_continue_example.sh - Continue in Loop

```
#!/bin/bash
for item in {0..99}
do
    if (( item == 3 )); then
        continue
    fi

    echo "$item"
done
```

Demonstrates using continue to skip an iteration.

Lab Exercises

Lab 13.1: Skip Range

File: 16_skip_range.sh

```
#!/bin/bash
# TO DO: Print numbers from 0 to 99, skipping numbers 20 to 80

for ((item=0; item<100; item++))
do
    # TO DO: Skip numbers 20 to 80
    your code here

    # TO DO: Print the current number
    your code here
done
```

Lab 13.2: Sales Tracker v2

File: 08_sales_tracker_v2.sh

```
#!/bin/bash
# TODO: Create a sales tracker that will ask the user how many items will be calculated,
# then ask for the cost and quantity of each item, and finally print the total number of items and the
total amount.

# TODO: Ask the user how many items will be calculated
your code here

total=0
packs=0

# TODO: Use a for loop to ask for more than one cost and count
for ((x=1; x<=input_count; x++))
do

    # TODO: Ask for the cost and quantity of each item
    YOUR CODE HERE
    YOUR CODE HERE

    # TODO: Calculate the total cost and total number of items
    YOUR CODE HERE

    #TODO: Add the current item cost and quantity to the total
    YOUR CODE HERE

    #TODO: Add the current item quantity to the total number of items
    YOUR CODE HERE
done

# TODO: Print the total number of items and the total amount
YOUR CODE HERE
```

Topic 14: Case Statements

Learning Objectives

By the end of this topic, you should be able to:

1. Understand the syntax and structure of case statements.
2. Use case statements for pattern matching.
3. Implement case statements in menu-driven programs.
4. Handle multiple patterns and wildcard matching.
5. Compare case statements with `if-elif-else` chains.

What Is a Case Statement?

The `case` statement in Bash is a powerful conditional construct that allows you to match a value against multiple patterns. It is cleaner and more readable than using multiple `if-elif` statements when checking a single variable against many possible values.

Syntax

```
case expression in
    pattern1)
        commands
        ;;
    pattern2)
        commands
        ;;
    patternN)
        commands
        ;;
    *)
        default commands
        ;;
esac
```

Sample Scripts

01_case_basic.sh - Basic Case Statement

```
#!/bin/bash
# Demonstrates a basic case statement for the days of the week

read -p "Enter a day (Monday-Sunday): " day

case $day in
    Monday)
        echo "Start of the work week!"
        ;;
    Tuesday|Wednesday|Thursday)
        echo "Middle of the work week."
        ;;
    Friday)
        echo "TGIF! The weekend is coming!"
        ;;
    Saturday|Sunday)
        echo "It's the weekend! Time to relax!"
        ;;
    *)
        echo "Invalid day entered."
        ;;
esac
```

Lab Exercises

Lab 14.1: Restaurant Menu System

File: 17_case_restaurant.sh

```
#!/bin/bash
# Demonstrates a case statement in a restaurant menu system

echo "=== Welcome to Bash Bistro ==="
echo "1. Pizza - $12.99"
echo "2. Burger - $9.99"
echo "3. Pasta - $11.99"
echo "4. Salad - $7.99"
echo "5. Drink - $2.99"
echo "===== "

read -p "Enter your choice (1-5): " choice

case $choice in
    Your code here
    Your code here
    5)
        case
            Your code here
        esac
esac
```